LAPORAN PRAKTIKUM

Aplikasi Berbasis Platform

MODUL 1,2,3

Running Modul, Pengenalan Flutter, Pengenalan Dart

Disusun oleh:

NAUFAL GERALDO PUTRA P.

2311102154

S1 IF-11-04

Dosen Pengampu:

Cahyo Prihantoro, S.Kom., M.Eng

PROGRAM STUDI S1 INFORMATIKA

FAKULTAS INFORMATIKA

INSTITUT TEKNOLOGI TELKOM PURWOKERTO

2026/2027

LINK GITHUB: https://github.com/NGZ101/Praktikum-APB

GUIDED

1. Running Modul

1.1 Git

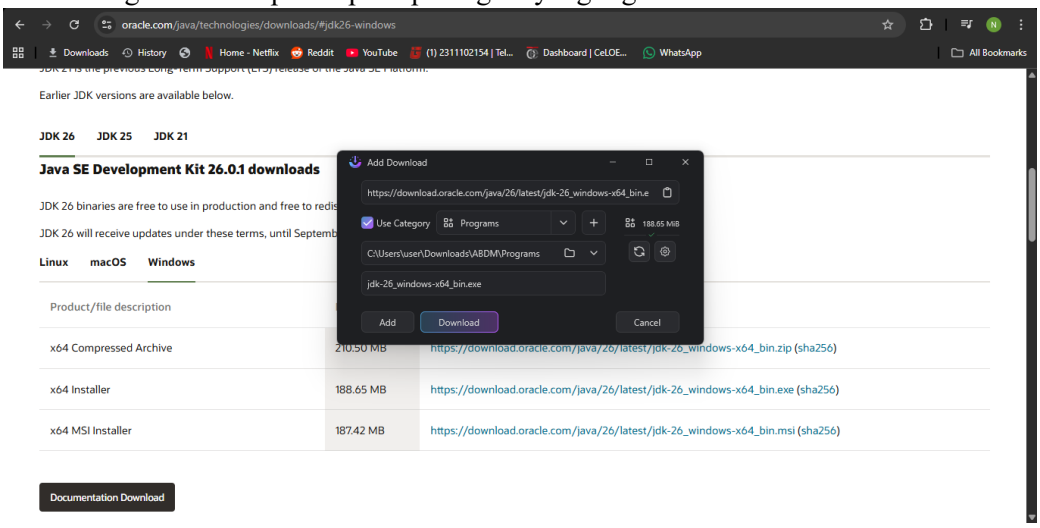
Git adalah salah satu sistem pengontrol versi (Version Control System) pada proyek perangkat lunak yang diciptakan oleh Linus Torvalds. Pengontrol versi bertugas mencatat setiap perubahan pada file proyek yang dikerjakan oleh banyak orang maupun sendiri. Git dikenal juga dengan distributed revision control (VCS terdistribusi), artinya penyimpanan database Git tidak hanya berada dalam satu tempat saja.

- Buka link berikut ini untuk mengunduh Git. https://git-scm.com/download/win
- Berikut adalah Git setelah download

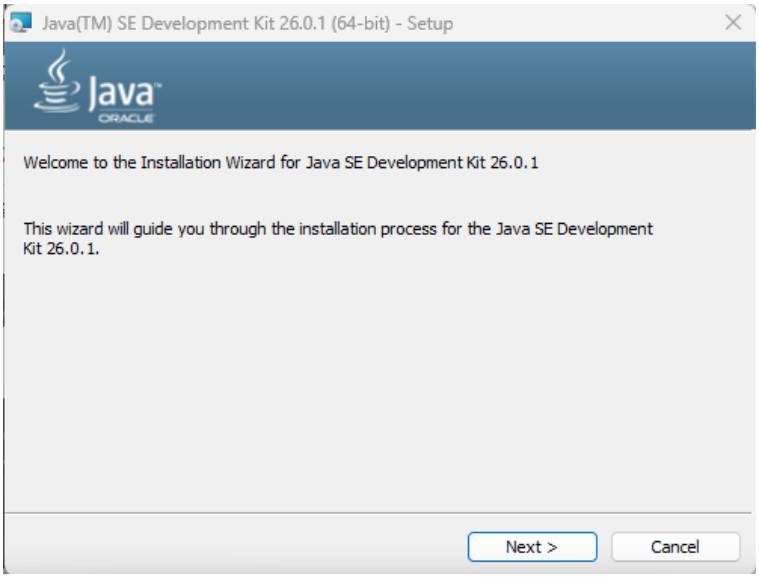
1.2 Instalasi JDK

JDK (Java Development Kit) merupakan perangkat yang digunakan untuk melakukan proses kompilasi dari kode java ke bytcode yang dapat dimengerti dan dapat dijalankan oleh JRE (Java Runtime Environment). Berikut merupakan tata cara dalam melakukan instalasi JDK:

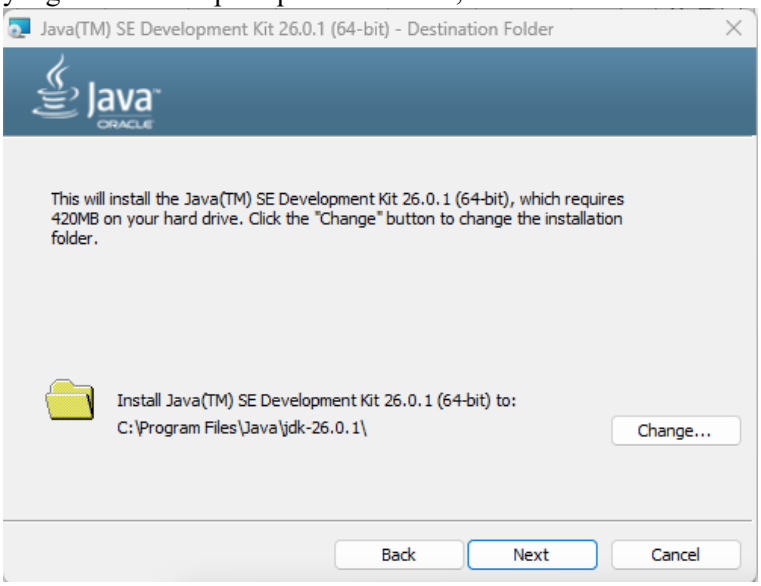
- buka link https://www.oracle.com/java/technologies/javase-jdk15-downloads.html untuk PEMROGRAMAN PERANGKAT BERGERAK 12 download, lalu pilih sesuai dengan sistem operasi pada perangkat yang digunakan.



- Buka file instalasi yang telah didownload, lalu klik next.

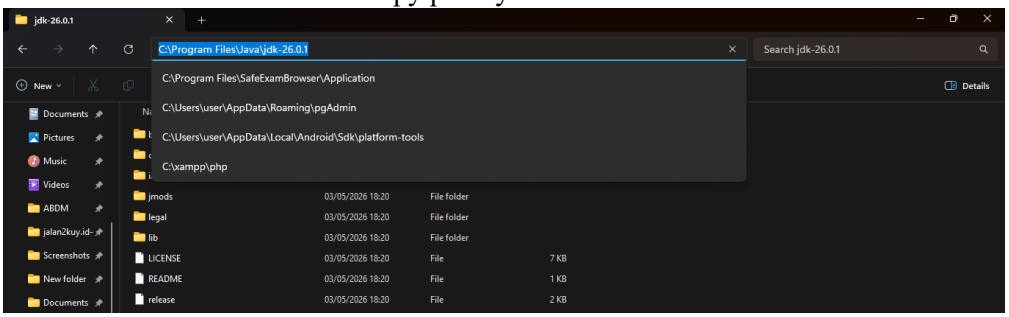


- Pilih folder path tempat menyimpan instalasi JDK, direkomendasikan sesuai dengan yang telah tertera pada proses instalasi, lalu klik next.

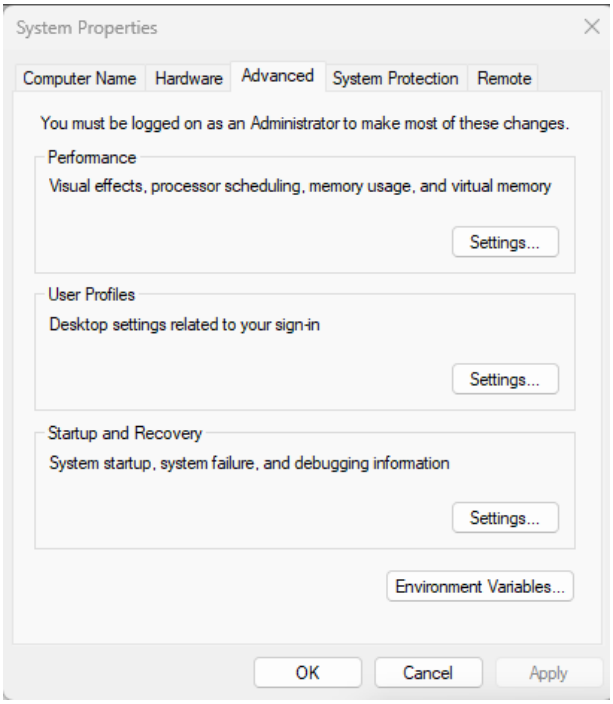


- Setelah proses instalasi selesai klik close.

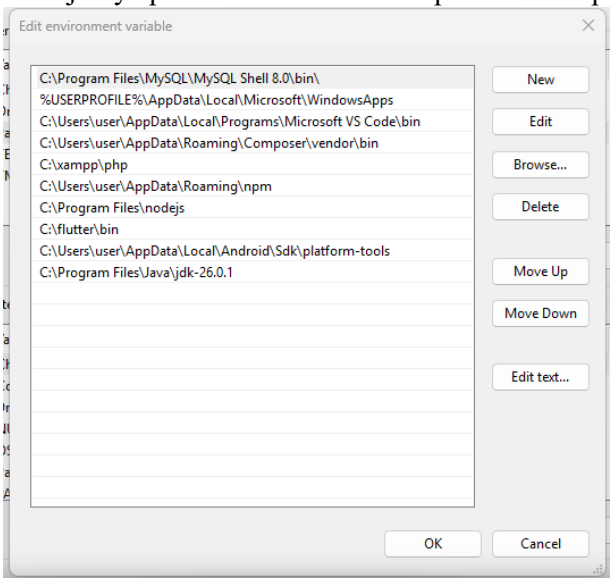
- Akses folder instalasi tadi lalu copy pathnya.



- Search envronment variable, lalu buka aplikasi tersebut dan pada tab advance klik Environment Variables.



- Selanjutnya pada user variables klik path dan add path ke jdk

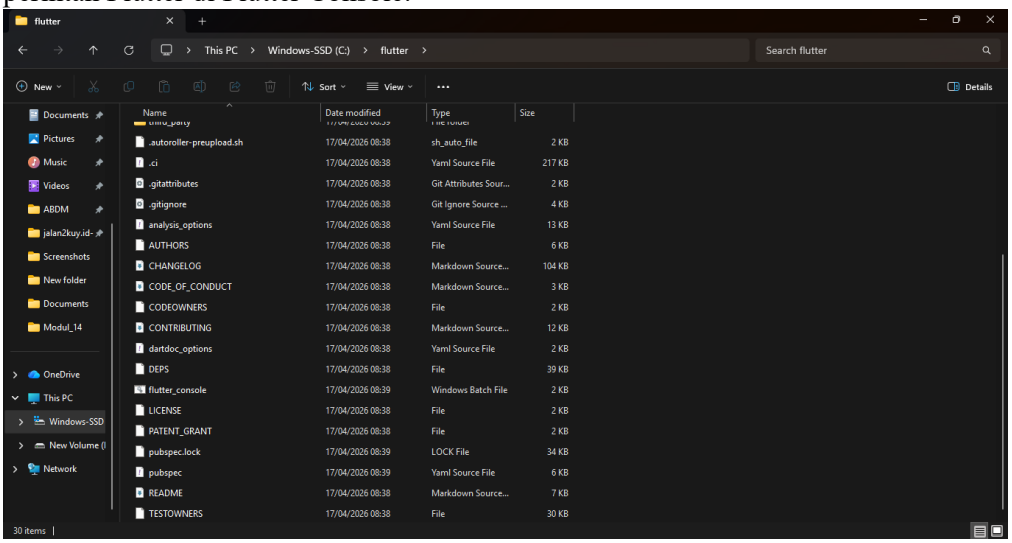


1.3 Instalasi Flutter SDK

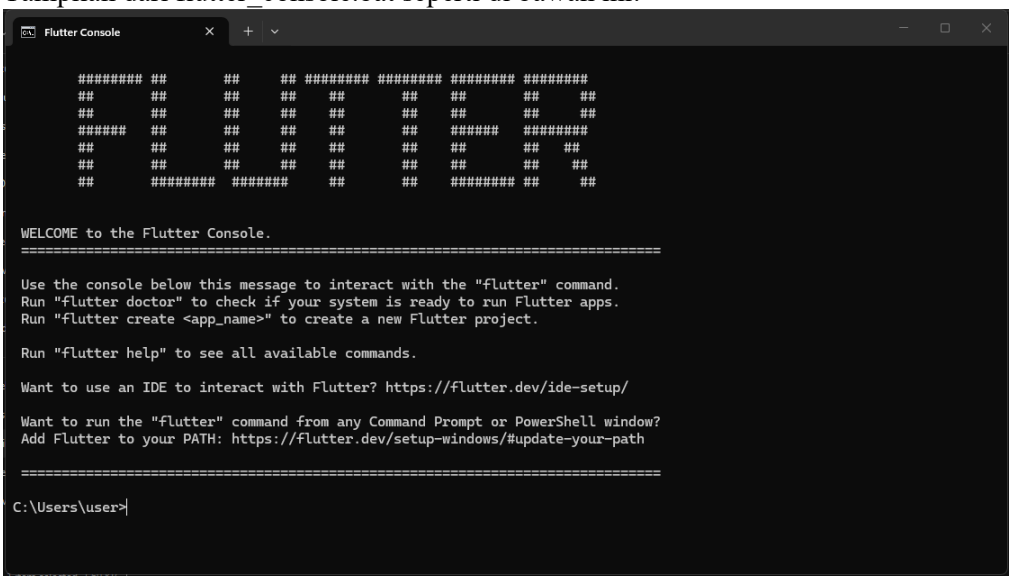
- Unduh Flutter SDK pada link dibawah ini, dan pastikan unduh versi yang stabil dan yang terbaru dari Flutter. Sesuaikan juga dengan sistem operasi yang dimiliki. Flutter

SDK dapat diunduh melalui link:
https://flutter.dev/docs/development/tools/sdk/releases

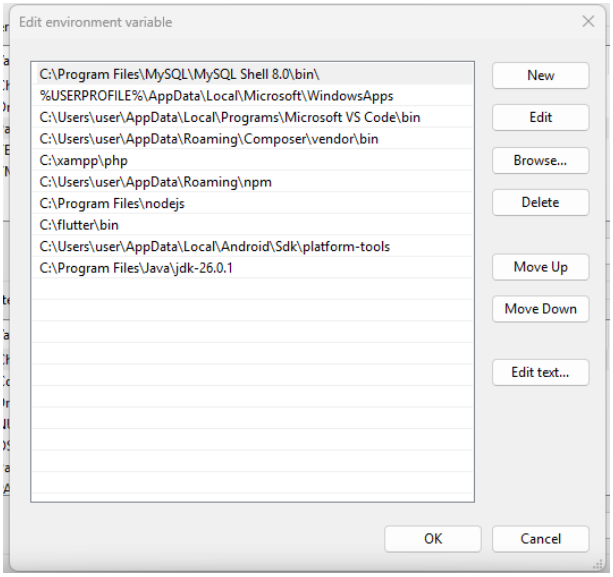
- Ekstrak berkas zip dan tempatkan folder flutter pada lokasi instalasi yang diinginkan untuk Flutter SDK, misalnya C:\Development. Catatan: Jangan pasang Flutter di direktori seperti C:\Program Files atau yang membutuhkan hak istimewa seperti administrator.
- Temukan berkas flutter_console.bat di dalam direktori flutter tersebut. Mulai dengan klik dua kali atau jalankan script tersebut dan Anda sekarang siap untuk menjalankan perintah Flutter di Flutter Console.



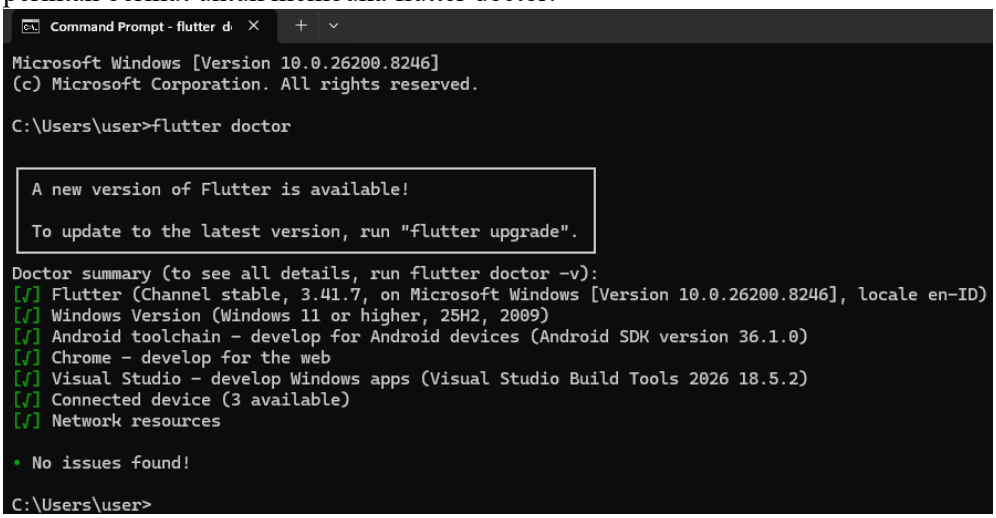
- Tampilan dari flutter_console.bat seperti di bawah ini:



- Berikut path flutter pada environment variable



- Flutter Doctor
  Flutter doctor merupaka perintah untuk mengecek kelengkapan framework flutter yang akan digunakan, seperti versi, Android SDK yang digunakan, iOS SDK yang digunakan (tersedia di MacOS), perangkat yang terhubung, dan lain-lain). Jalankan perintah berikut untuk membuka flutter doctor:



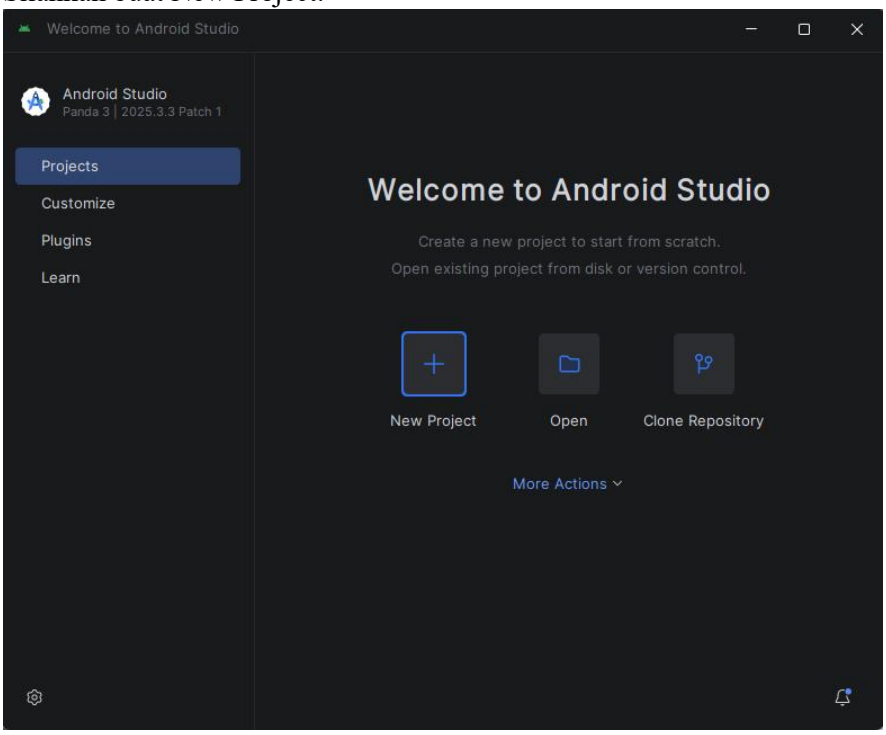
  Perintah ini memeriksa environment Anda dan menampilkan laporan ke jendela terminal. Pada Flutter SDK sudah terdapat Dart SDK, jadi Anda tidak perlu menginstal Dart secara terpisah. Periksa output dengan cermat untuk perangkat lunak lain yang mungkin perlu Anda instal atau melakukan sesuatu lebih lanjut (ditunjukkan dalam teks tebal).

1.4 Instalasi Android Studio

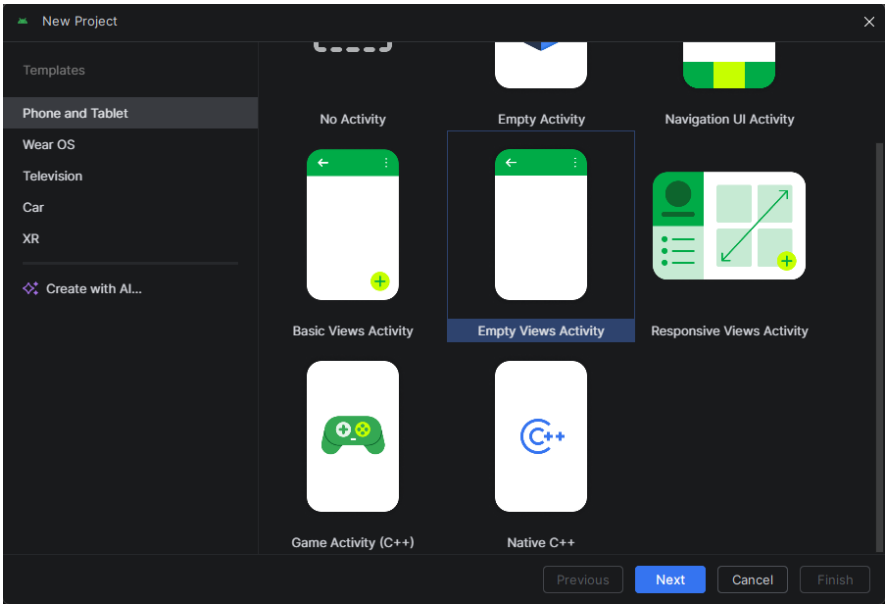
Android Studio merupakan IDE resmi dalam membangun aplikasi berbasis Android, proses instalasinya juga sederhana sehingga kita dapat dengan mudah membangun aplikasi menggunakan Android Studio.

- Download aplikasi Android Studio terbaru pada https://developer.android.com/studio. Tersedia untuk Sistem Operasi Windows, Mac dan Linux.
- Berikut setelah download Android Studio

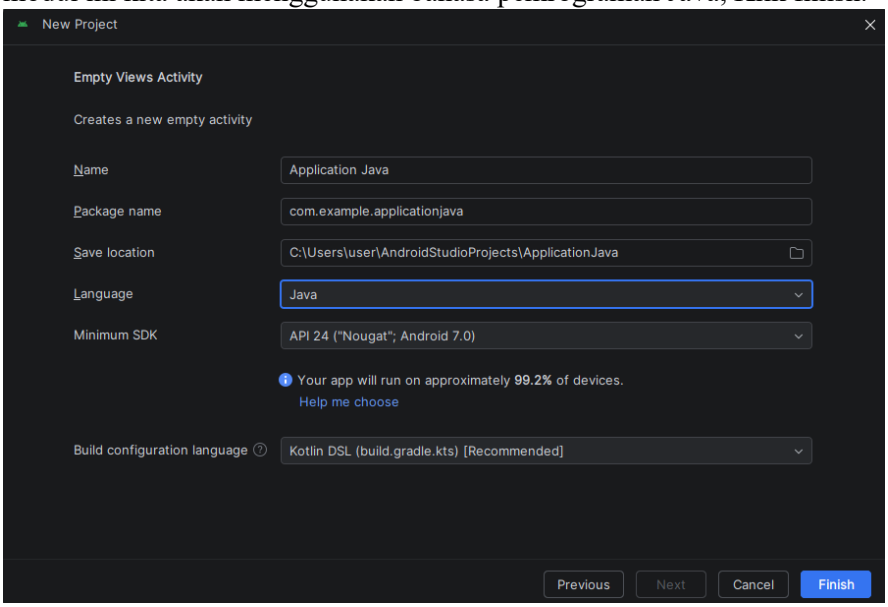
- Silahkan buat New Project.



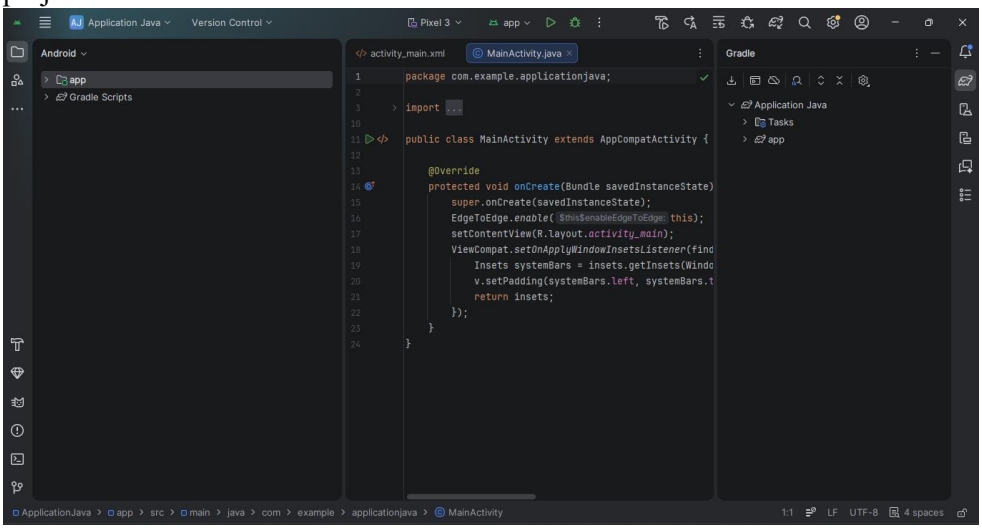
- Pilih Empty Activity lalu klik next.



- Silahkan masukan nama aplikasi serta bahasa pemrograman yang diinginkan, pada modul ini kita akan menggunakan bahasa pemrograman Java, Klik finish.



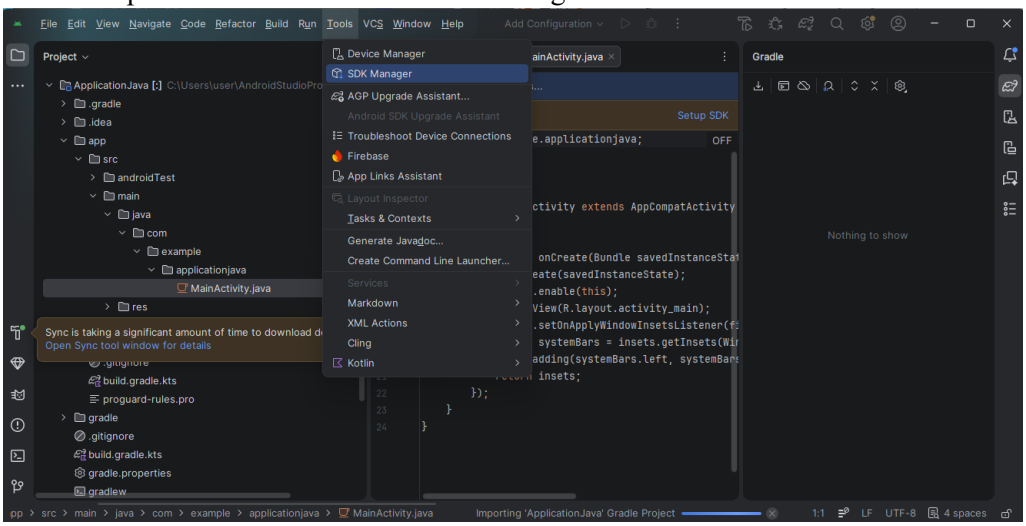
- Berikut merupakan tampilan awal pada aplikasi Android Studio setelah membuat projek baru.



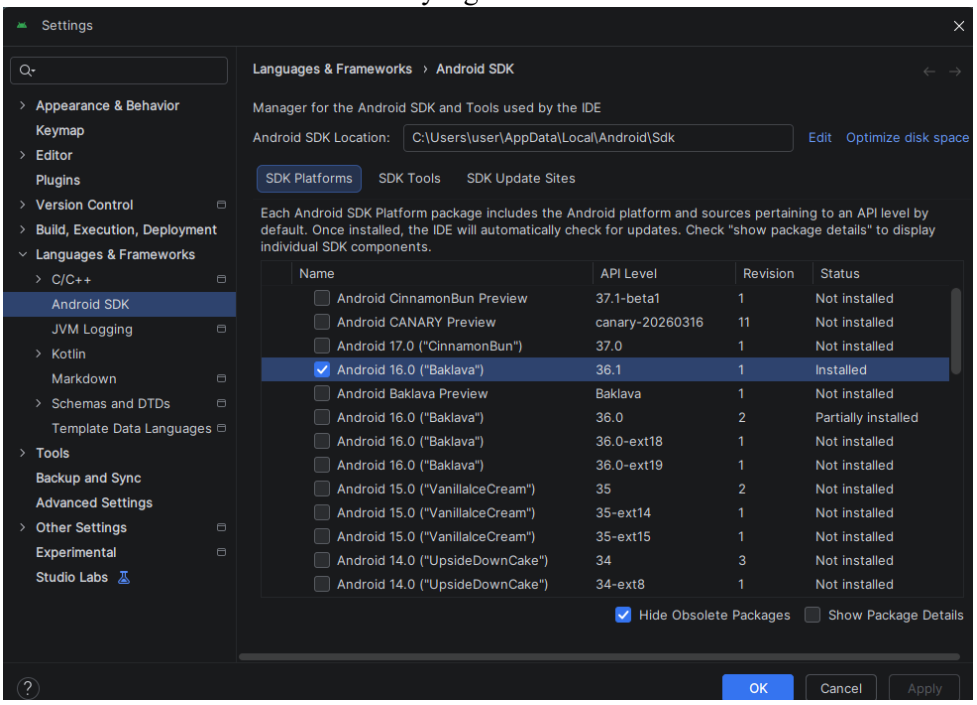
1.5 Instalasi SDK Android

SDK (Standart Development Kit) merupakan kumpulan dari beberapa alat, komponen, juga platform untuk mengembangkan aplikasi berbasis android. SDK wajib ada pada Android Studio. Berikut merupakan cara instalasi SDK:

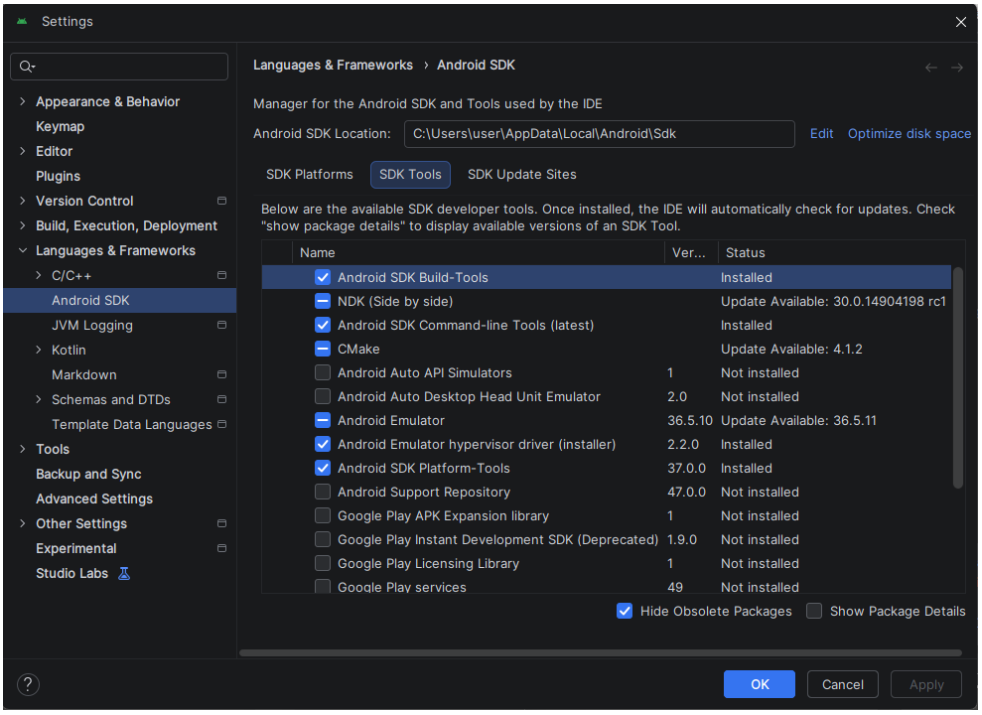
- Klik tools pada menu bar lalu Klik SDK Manager.



- Pada SDK Platform checklist SDK yang kamu butuhkan



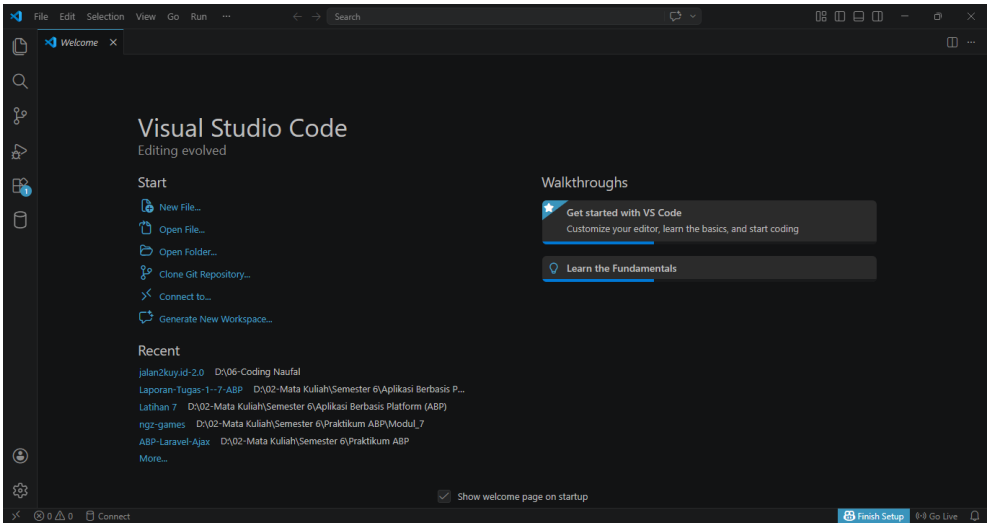
- Pada SDK Tools checklist tools yang kamu butuhkan



1.6 Instalasi Visual Studio Code

Pada kali ini, kita akan menggunakan visual studio code sebagai editor untuk membuat aplikasi menggunakan Flutter. Berikut langkah-langkah instalasinya:

- Download visual studio code di https://code.visualstudio.com/Download
- Berikut setelah download

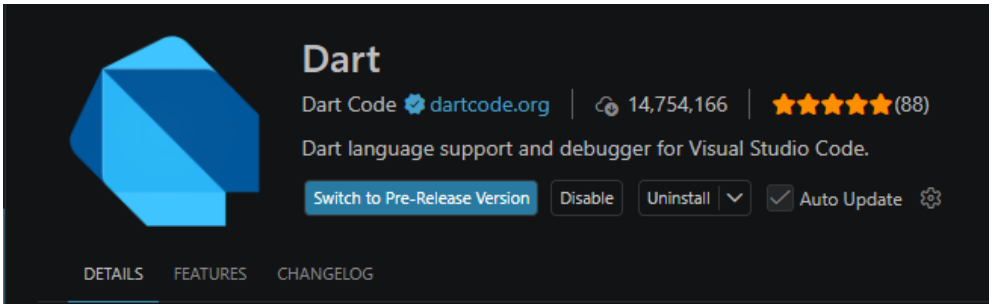


1.7 Instalasi Extension Visual Studio

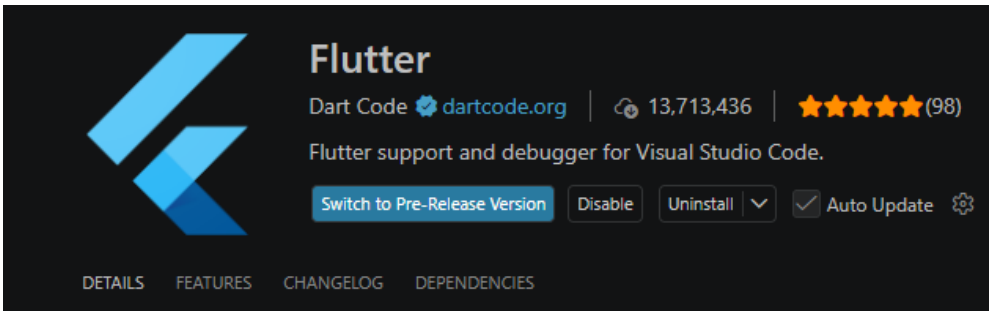
Sebelum menggunakan visual studio code, sangat diperlukan untuk menginstall extension yang nantinya sebagai pendukung ketika membuat aplikasi menggunakan Flutter. Berikut extension yang perlu diinstal:

- Dart

  Extension ini perlu diinstal karena untuk menjalankan Flutter, kita perlu menginstal Dart agar aplikasi yang dibuat bisa berjalan dan tidak terjadinya error.



- Flutter
  Extension ini sangat penting ketika ingin membuat aplikasi menggunakan Flutter di visual studio code.

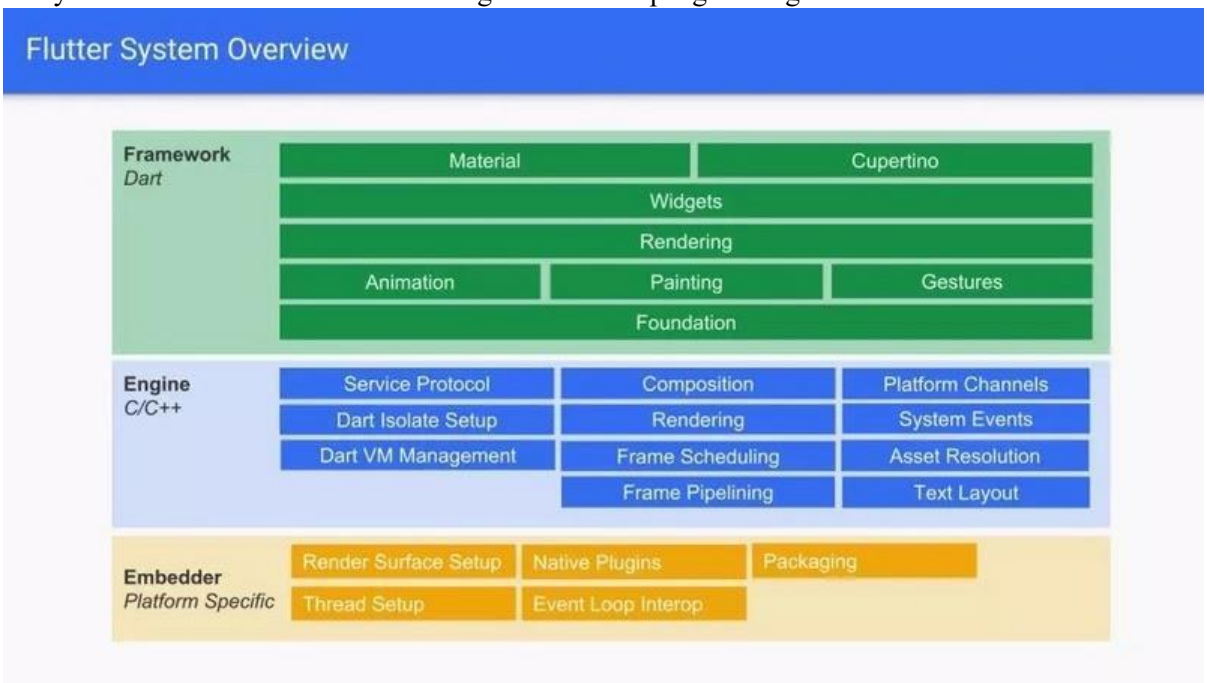


2. Pengenalan Flutter

2.1 Apa itu Flutter

Flutter ditulis menggunakan bahasa C, C++ dan Dart dengan Google's Skia Graphics Engine untuk user interface. Engine yang digunakan untuk produk ini dikenal seperti Google Chrome, Chrome OS, Chromium OS, Mozilla Firefox, Mozilla Thunderbird, Android, Firefox OS dan sekarang Flutter.

Flutter berjalan menggunakan Dart Virtual Machine (VM) di sistem operasi Windows, Linux, dan macOS. Dart VM menggunakan kompilasi kode just-in-time (JIT) yang menyediakan fitur hot-reload untuk menghemat waktu pengembangan



Flutter API menggabungkan beberapa widget sesuai dengan kebutuhan pada aplikasi. Konsep widget tree pada dasarnya adalah implementasi widget yang di dalamnya terdapat

widget lain yang mewakili komponen antarmuka. Widget bisa stateless atau stateful dan perbedaan dari keduanya sesuai dengan status widget itu sendiri. Hal tersebut berguna untuk membantu mengelola status aplikasi.

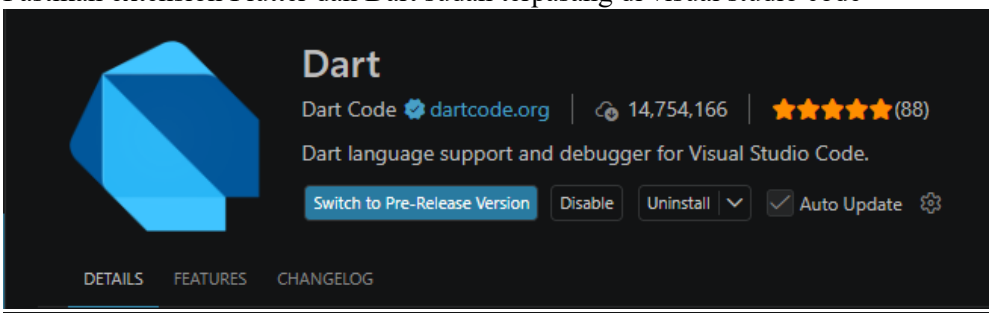
2.2 Arsitektur Flutter

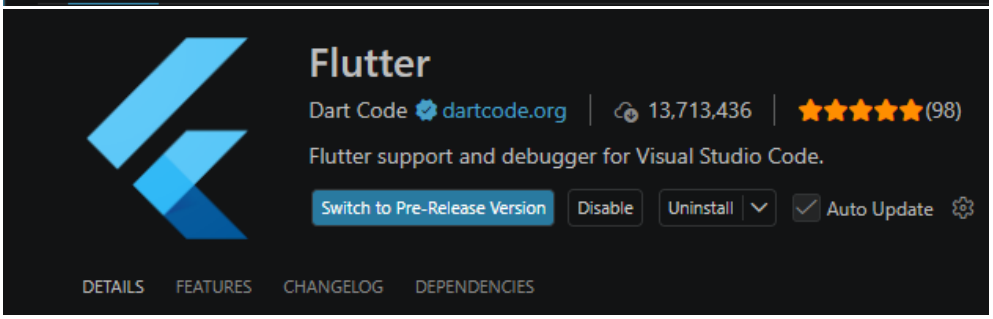
Selain dari konsep inti yang dibahas pada sub bab sebelumnya, Flutter mempunyai arsitektur dasar yang nantinya dapat di terapkan pada aplikasi dan mengelola statusnya dengan mudah. Arsitektur yang digunakan pada Flutter disebut Business Logic Component (BLOC). Pada dasarnya, hall ini melalui pendekatan ketika terjadinya suatu event dan menghandle perubahan state pada aplikasi. BLOC merupakan pendekatan yang baik untuk memisahkan logika bisnis dari antarmuka. Ide intinya yang digunakan pada arsitektur BLOC adalah simplicity, scalability, dan testability, dan semua tujuan ini pasti dicapai dalam arsitektur BLOC.

2.3 Hello World Pada Flutter Pengenalan Dart

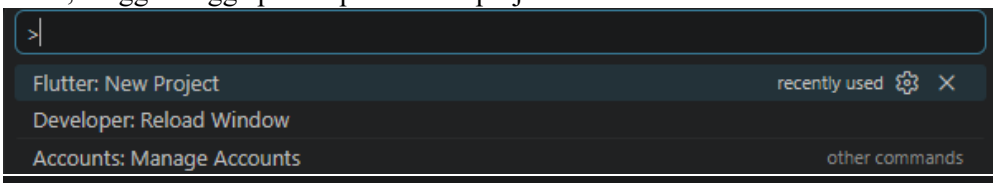
Pada pengenalan Flutter kali ini, kita akan membuat Hello World sebagai permulaan Ketika menggunakan Flutter. Ikuti langkah berikut untuk membuat Hello World di Flutter:

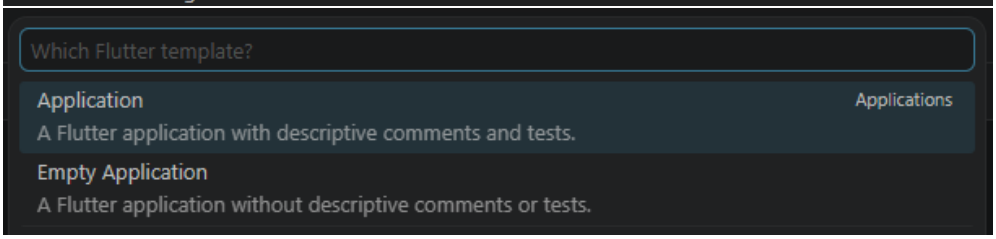
- Buka visual studio code kalian untuk membuat project Flutter
- Pastikan extension Flutter dan Dart sudah terpasang di visual studio code



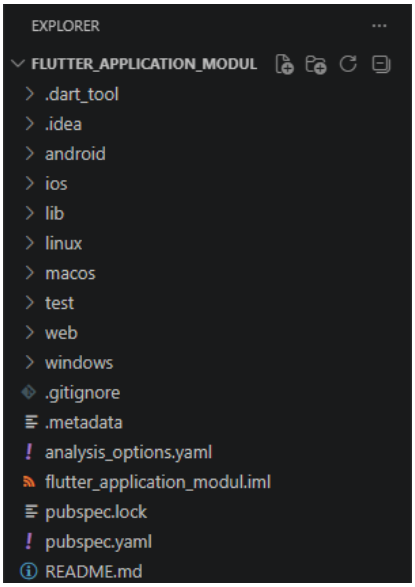


- Lalu buat project Flutter pada menu View -> Command Palette -> lalu pilih Flutter: New Project -> pilih Application untuk membuat aplikasi dengan Flutter -> pilih folder yang ingin dijadikan sebagai project Flutter -> lalu beri nama project dan tekan Enter, tunggu hingga proses pembuatan project Flutter selesai

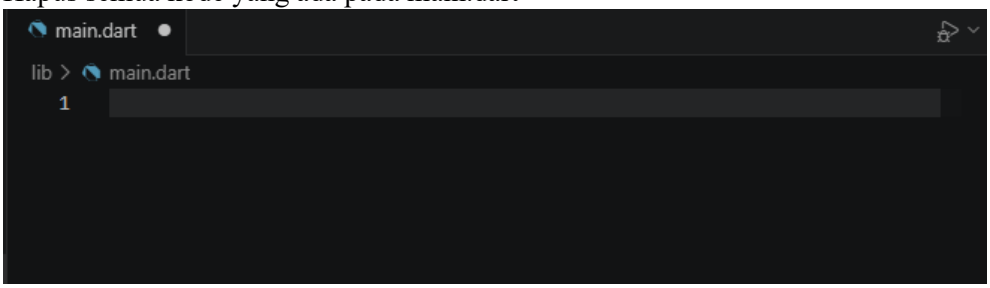




- Ketika project selesai dibuat, maka tampilan folder akan seperti ini



- Buka file main.dart untuk memulai membuat Hello World
- Hapus semua kode yang ada pada main.dart



- Lalu tambahkan barisan kode berikut untuk mebuat Hello World

```
// ignore_for_file: prefer_const_constructors

import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({Key? key}) : super(key: key);

  // This widget is the root of your application.
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: "Hello World",
      home: const MyHomePage(title: "Flutter Hello World
Page"),
    );
  }
}
```

```
class MyHomePage extends StatefulWidget {
  const MyHomePage({Key? key, required this.title}) :
super(key: key);

  final String title;

  @override
  State<MyHomePage> createState() => _MyHomePageState();
}

class _MyHomePageState extends State<MyHomePage> {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text(widget.title)),
      body: Center(child: Text('Hello World')),
    );
  }
}
```

3. Pengenalan Dart

3.1 Pengenalan Dart

Untuk belajar flutter, tidak perlu terlalu fasih untuk mempelajari bahasa dart. Terdapat fundamental yang perlu dipelajari seperti variable, statement control, looping, array, fungsi, dsb. Karakteristik bahasa dart mirip dengan bahasa C ataupun Java. Wajib menggunakan titik koma diakhir codingan.

3.1.1 Variable

Untuk penggunaan variable di dart, terdapat beberapa cara, yaitu dengan var, type annotation dan multiple variable.

```
// var
var <variable_name>;
var <name> = <expression>;

// type annotation
<type> <variable_name>;
<type> <name> = <expression>;

// multiple variable
<type> <var1,var2...varN>;
```

Variable primitif yang tersedia di dart :

1. Integer
2. Double
3. String
4. Boolean

3.1.2 Statement Control

Terdapat beberapa cara untuk mendeklarasikan statement control, yaitu if, if else, if else if, switch case.

IF Statement

```
// IF STATEMENT
```

```
if(condition){
        // statements
}
```

IF ELSE Statement

```
// IF ELSE STATEMENT
if(condition){
        // statements
} else {
        // statements
}
```

IF ELSE IF Statement

```
// IF ELSE IF STATEMENT
if(condition1) {
// statement(s)
}
else if(condition2){
 // statement(s)
}
.
.
else if(conditionN){
 // statement(s)
}
else {
 // statement(s)
}
```

SWITCH CASE Statement

```
// SWITCH CASE
switch(expression){
 case value1: {
   // statements
   }
      break;
 case value2: {
   // statements
   }
      break;
 default: {
   // statements
   }
      break;
}
```

3.1.3 Looping

Secara umum, terdapat dua cara untuk melakukan looping di dart, yaitu menggunakan for loop dan while loop.

For Loops

```
for (initial_count_value; termination-condition; step) {
  //statements
```

```
}
```

While Loops

Gunakan while loop saat kondisinya tidak tahu kapan perulangan akan berhenti, contohnya sediakan input angka hingga user menginput tanda "-".

```
while (expression) {
  // Statement(s) to be executed if expression is true
}
```

3.1.4 List

Secara umum, kumpulan banyak data dalam satu variable disibut array. Tetapi beberapa Bahasa pemrograman menyebutnya dengan list, termasuk bahasa dart ini. List memiliki 2 tipe, yaitu Fixed Length List dan Growable List.

Fixed Length List

Dari namanya bisa diketahui bahwa tipe list ini memiliki panjang index yang tetap dan tidak dapat bertambah banyak.

```
// Mendeklarasikan list
var list_name = new List(initial_size);

// Menginisialisasikan list
list_name[index] =  value;

// Contohnya
var newList = new List(3);
newList[0] = 12;
newList[1] = 13;
newList[2] = 11;
```

Growable List

Gunakan growable list apabila memiliki banyak object yang tidak menentu atau banyaknya object yang terus bertambah.

```
// Mendeklarasikan list
var list_name = new List();

// Menginisialisasikan list
list_name[index] = value;

// Contohnya
var newList = new List(3);
newList[0] = 12;
newList[1] = 13;
newList[2] = 11;
```

3.1.5 Fungsi

Pada bahasa pemrograman yang mendukung Object Oriented Programming, fungsi atau prosedur memilki peranan yang sangat penting. Untuk menghasilkan kualitas kode yang sangat baik, programmer bisa menggunakan beberapa prinsip pemrograman yang umum digunakan seperti SOLID, KISS, YAGNI, dsb. Semua prinsip tersebut menjunjung tinggi separation of concern yang artinya setiap kodingan memiliki tanggung jawabnya sendiri dan mengurangi sebanyak mungkin boilerplate code.

Mendefinisikan Fungsi

```
void function_name() {
  //statements
}
```

Memanggil Fungsi

```
void main() {
 print(factorial(6));
}
```

Mengembalikan Nilai

Tambahkan return apabila anda mendefinisikan sebuah fungsi, contohnya ada pada codingan dibawah yang bisa mengembalikan nilai faktorial dari angka yang sudah ditentukan.

```
factorial(number) {
  if (number <= 0) {
    // termination case
    return 1;
  } else {
    return (number * factorial(number - 1));
    // function invokes itself
  }
}
```

Menambahkan Parameter

Fungsi memiliki scope yang terbatas, tentunya fungsi butuh input dari luar agar program didalamnya bisa memproses tugasnya.

```
factorial(number) {
  if (number <= 0) {
    // termination case
    return 1;
  } else {
    return (number * factorial(number - 1));
    // function invokes itself
  }
}
```

Pada fungsi diatas, number merupakan parameter. Variable diluar fungsi yang dibuat agar dapat digunakan didalam fungsi.